ASSIGNMENT 3 – TASK 5

Race Conditions in a Cinema Seat-Booking System

Programming in Java | CO3 | 6 Marks | Bloom's Level: K4 – Analyse

1. Problem understanding

A cinema booking operation checks whether a seat is free and then marks it booked. If two threads interleave between the check and update, both may observe the seat as free and both may report success. This is a race condition caused by a non-atomic check-then-act operation.

2. Exact critical section

```
if (!booked[seat]) {
    booked[seat] = true;
    bookingsMade++;
}
```

The check, state update and successful-booking counter increment must be protected as one atomic operation. Protecting only the assignment but not the check is insufficient.

3. Race condition interleaving

```
Thread A                        Thread B
--------                        --------
reads booked[4] == false
                                reads booked[4] == false
writes booked[4] = true
increments bookingsMade
                                writes booked[4] = true
                                increments bookingsMade
Both believe they booked seat 4.
```

The demo uses a CountDownLatch barrier after the unsynchronized check. This deliberately pauses contenders until all 100 have passed the free-seat test, then releases them together. It makes the race repeatable for demonstration. This barrier is test instrumentation; it is not part of a production booking algorithm.

4. Complete the Code – TODO 1

```
public synchronized void book(int seat) {
    if (!booked[seat]) {
        booked[seat] = true;
        bookingsMade++;
    }
}
```

The synchronized keyword makes the check-and-book operation atomic with respect to other synchronized operations on the same object. The supplied SeatBooking.java contains the completed assignment program and should print Seats sold: 10.

5. Synchronized method versus synchronized block

A synchronized instance method acquires the intrinsic monitor of this for the entire method. A synchronized(this) block acquires the same intrinsic monitor but only for the statements inside the block. Both prevent two threads from entering the protected critical section concurrently on the same object. A block is useful when validation or other work that does not touch shared mutable state can safely be done before acquiring the lock.

6. Intrinsic lock and static synchronization

Every Java object has an intrinsic monitor. A synchronized instance method locks the receiver object; synchronized(this) locks that same receiver. A static synchronized method locks the Class object. Static synchronization is appropriate only when multiple instances must coordinate access to class-wide shared state. For 20 independent screens, one instance lock per screen permits bookings on different screens to proceed concurrently.

7. Student A – experimental race evidence

The program runs the unsynchronized test ten times. Each trial creates 100 users competing for ten seats (ten contenders per seat). The latch coordinates the test so the race is reliably exposed. Record the actual counts printed by your own run. The synchronized method and block versions should each report exactly ten successful bookings.

8. Student B – 10,000 booking performance

The included timing test performs 10,000 attempts using ten worker threads and 10,000 distinct seats. It measures elapsed wall-clock time for the synchronized method and synchronized block. Since both implementations use the same intrinsic monitor and the protected work is small, measured differences may be small and noisy. Run multiple trials on the target computer, record raw values, and compare averages; do not treat one sample timing as a universal result.

9. Joint engineering decision: 20 screens

Use independent booking-state objects and locks per screen. This prevents duplicate booking on each screen while allowing different screens to proceed concurrently. A synchronized method is easiest to maintain and is suitable when the method contains only the short critical operation. A synchronized block can reduce lock hold time when safe non-critical work exists outside it. Avoid a single global lock across all screens unless a shared cross-screen rule requires it.

10. Alternative concepts

AtomicBoolean[]: create one AtomicBoolean per seat and book with compareAndSet(false, true). Only one caller succeeds for a seat. ReentrantLock: acquire lock, check and update, then unlock in finally; tryLock with a timeout can provide bounded waiting but requires a policy for unsuccessful attempts. ConcurrentHashMap: use putIfAbsent(seatId, booking) to atomically claim a seat. Each is O(1) expected time per attempt; under contention, waiting/retries affect practical latency.

11. Complexity and space

For s seats, the boolean[] uses O(s) space. Each booking attempt performs O(1) expected work; b attempts therefore require O(b) total work apart from contention. Synchronized method and block use the object's intrinsic monitor, with O(1) additional synchronization state per screen. AtomicBoolean[] uses O(s) references and per-seat atomic objects. A ConcurrentHashMap uses O(s) storage for booked seats plus map overhead. Lock contention increases waiting time and reduces throughput but does not change the basic O(1) critical operation.

12. Conclusion

The double-booking defect occurs because the check-then-book sequence is not atomic. Synchronizing the complete operation prevents the race. For a multi-screen cinema, independent per-screen locks provide correctness without unnecessarily blocking unrelated screens. AtomicBoolean.compareAndSet and other atomic map/lock approaches are valid alternatives.

This report intentionally contains no student-specific details. Attach your own screenshots and repeated timing measurements before submission.